Spécifications

Dans cette itération, vous allez implémenter les fonctionnalités décrites ci-dessous.

Problématique métier à implémenter

Cette application gère un **tableur** d'une taille de `ROWS x COLUMNS` (`ROWS` et `COLUMNS` sont des constantes qui valent respectivement 10 et 4 dans l'exemple ci-dessous).

L'utilisateur peut modifier le contenu d'une **cellule**.

Une cellule a un **contenu** (textuel) et une **valeur** (calculée).

Contenu textuel d'une cellule

Le contenu d'une cellule est **un texte** qui, soit représente une ***valeur littérale***, soit est une ***expression***.

Valeur littérale

Une valeur littérale peut être :

- un nombre décimal (avec des virgules). Exemples : `54`, `84,04`, ...
- un booléen. Exemples : `true`, `False`, en majuscules ou minuscules
- du texte. Exemples : `Toto`, `Il fait chaud`, ...

Expression

Dans le cas d'une expression, le texte de la cellule commence alors par le caractère `=` et est suivi d'une expression représentant un calcul arithmétique ou logique.

Une expression peut comprendre :

- des *valeurs littérales*
- des *références* vers d'autres cellules (`A1`, `D8`, ...)
- des *opérateurs arithmétiques* (caractères `+`, `-`, `*`, `/`)
- des *opérateurs logiques* (mots clés `and`, `or`, `not`, en majuscules ou minuscules)
- des *opérateurs de comparaisons* utilisés dans les expressions logiques (`>`, `>=`, `<`, `<=`, `=`, `!=`)

Exemples d'expressions complexes à considérer :

- `= 5 + 3 * 2`, expression arithmétique avec opérateurs arithmétiques et opérandes littérales.
- `= 5 > 3 and 2 > 1 or false`, expression logique avec opérateurs logiques et des opérandes qui sont des opérateurs de comparaison et une valeur littérale.
- `= 5 + 2 > 6`, expression logique avec une opérateur de comparaison comportant un opérande qui est une expression arithmétique.
- `= B2 + 3 * 5 + C4`, expression arithmétique avec deux références vers les cellules `B2` et `C4`.

Valeur (calculée) d'une cellule

Chaque cellule (non vide) a une **valeur**. Celle-ci est calculée comme suit.

Valeur littérale

Selon le type de valeur littérale contenue dans la cellule, la valeur calculée est égale :

- au contenu lui-même s'il s'agit d'un texte
- à la valeur numérique correspondante si le texte représente un nombre : le texte `54` a comme valeur `54`, etc
- à la valeur logique correspondante si le texte représente un booléen : le texte `True` a comme valeur `true`, etc

Valeur d'une expression

La valeur calculée d'une expression est le résultat du calcul représenté par l'expression.

- L'expression `= 5 + 3 * 2` a comme valeur `11`. Remarquez la priorité de l'opérateur `*` sur l'opérateur `+`.
- L'expression `= 5 > 3 and 2 > 1 or false` a la valeur `true`.
- L'expression `= B2 + 3 * 5 + C4` a **la valeur de la cellule `B2` + `15` + la valeur de la cellule `C4`** (pour autant que `B2` et `C4` contiennent des valeurs numériques).

> **Important**
>
> La valeur d'une expression dépendant d'autres cellules doit être recalculée si le contenu d'une cellule référencée change.

Priorité des opérateurs

Les opérateurs sont évalués selon l'ordre de priorité suivant (du plus prioritaire au moins prioritaire) :

1. Opérateurs arithmétiques multiplicatifs (`*` et `/`)
2. Opérateurs arithmétiques additifs (`+` et `-`)
3. Opérateurs de comparaison (`>` , `>=` , `<` , `<=` , `=` , `!=`)
4. Opérateur logique `not`
5. Opérateur logique `and`
6. Opérateur logique `or`

Les opérateurs de même niveau de priorité sont évalués de gauche à droite.

Exemples :

- `= 2 + 3 * 4` donne `14` car `*` est plus prioritaire que `+`
- `= 8 / 2 * 4` donne `16` car à priorité égale, on évalue de gauche à droite
- `= 5 > 3 and not 2 > 4 or true` donne `true` car l'évaluation se fait dans cet ordre : `(5 > 3) and (not (2 > 4)) or true`

Cas d'erreur

Deux types d'erreurs peuvent se produire lorsque l'utilisateur construit une expression : **erreur syntaxique** ou **erreur à l'évaluation**

Erreur syntaxique

Une erreur de syntaxe survient lorsque l'expression n'est pas bien formée.

La **valeur** d'une cellule présentant une erreur de syntaxe est positionnée à **SYNTAX ERROR**.

Exemples :

- `=5+3*` : pas d'opérande de droite pour l'opérateur `*`
- `=true + 5` : types incompatibles (valeur littérale booléenne et valeur littérale numérique)
- `=toto + machin` : on ne vous demande pas de gérer les opérations sur les chaînes de caractères

> **Info**
>
> En Excel, la concaténation se fait via l'opérateur `&` (et pas `+`) et les chaînes littérales de l'expression doivent être entre *double quotes* (`="toto"&"machin"`).

Erreur à l'évaluation

Plusieurs types d'erreurs peuvent survenir lorsque le système calcule la valeur d'une cellule.

La **valeur** d'une cellule est positionnée à **#VALEUR** *si l'une des deux erreurs suivantes se produit* :

- *erreur de type* : l'expression `= B2 + 3 * 5 + C4` déclenche une erreur de type si la valeur de `B2` ou de `C4` ne sont pas numériques
- *division par zéro* : l'expression `= B2 / C3` produit une erreur de division par zéro si la valeur de `C3` vaut `0`.

La **valeur** de chaque cellule concernée par des références circulaires est positionnée à **#CIRCULAR REF**.

Il y a une référence circulaire si, par exemple, la cellule `A1` référence une cellule `A2` qui référence `A1`, ou si `A1` référence `A2` qui référence `B1` qui référence... qui référence `A1`.

> **Remarque**
>
> L'erreur de *type incompatible* est détectée à l'analyse syntaxique uniquement pour les valeurs littérales. Par exemple, `=true + 5` est une erreur de syntaxe car on peut immédiatement voir qu'on essaie d'additionner un booléen avec un nombre.
>
> En revanche, une expression comme `=B4 and B6` est syntaxiquement correcte car on ne peut pas connaître le type des valeurs contenues dans B4 et B6 lors de l'analyse syntaxique. Si B4 ou B6 contient une valeur non booléenne, ce sera détecté comme une erreur d'évaluation (et non de syntaxe) et la cellule affichera #VALEUR.

Comportements purement visuels

- Le tableur doit utiliser le composant `SpreadsheetView` de la librairie `ControlsFX`.
- La **barre d'édition de cellule** doit toujours montrer le **contenu** (textuel) de la cellule sélectionnée (et rien si aucune cellule n'est sélectionnée).
- Il est possible d'**éditer le contenu d'une cellule** soit en cliquant deux fois sur celle-ci soit via la barre d'édition de cellule.
- On montre la **valeur calculée** d'une cellule si elle n'est pas en édition et le **contenu** si elle est en mode édition.
- Si la valeur calculée est numérique et que le nombre a une partie décimale égale à zéro ou très proche de zéro (<= 0.001), il doit s'afficher comme un entier. Par exemple `5.0` et

`5.001` s'affichent `5`. Sinon, le nombre s'affiche avec deux chiffres décimaux après la virgule. Par exemple `5.34567` s'affiche `5.35`.

Démo



Design Patterns à mettre en oeuvre

Interpreter

Une expression contenue dans une cellule va être représentée sous forme d'un arbre : la racine de cet arbre correspond à l'expression complète et chaque noeud enfant de l'arbre correspond à une sous-expression (et est donc lui-même un sous-arbre). Un noeud terminal (qui n'a pas d'enfant) correspond à une valeur littérale ou à une référence vers une autre cellule.

Pour obtenir la valeur calculée de la cellule, il faut calculer la valeur de chaque enfant et composer le résultat du parent en fonction de celui de ses éventuels enfants (par exemple, si le noeud racine représente l'opérateur `+`, la valeur calculée de la cellule est la somme des valeurs calculées de ses opérandes. Chaque opérande est un noeud enfant du noeud racine).

Etudiez et mettez en oeuvre le *design pattern* ***Interpreter*** qui sert justement pour cette problématique. Il existe d'autres méthodes pour arriver au même but mais pour ce projet, vous devez utiliser le *design pattern* ***Interpreter***.

Builder

Pour construire l'arbre de l'expression à partir d'une String, il faut décomposer cette String en plusieurs morceaux et reconnaître les différents éléments : valeurs littérales, opérateurs, références vers d'autres cellules.

Ensuite, il faut analyser si la grammaire est correcte (par exemple, un opérateur `+` doit avoir deux opérandes de types numérique ou référence,...)

Il y a donc plusieurs étapes (complexes) à effectuer pour construire l'expression. Dans ce genre de situation, on confie la construction de l'objet (ici une *Expression*) à une ou plusieurs classes spécialisées.

Etudiez et mettez en oeuvre le *design pattern* ***Builder*** qui sert justement pour cette problématique. Il existe d'autres méthodes pour arriver au même but mais pour ce projet, vous devez utiliser le *design pattern* ***Builder***.

Méthodologie

Avant de vous lancer dans la programmation, vous devez établir un **premier diagramme de classes du domaine** (et pas un diagramme d'implémentation qu'on appelle aussi diagramme de classes logiciel) et le faire valider par votre chargé de cours.

Ensuite, procédez par étapes :

- Expressions numériques simples avec littéraux et opérateurs arithmétiques binaires (`+`, `-`, `*`, `/`)
- Expressions numériques avec références de cellule (`A1`, `D8`, ...)
- Expressions logiques avec opérateurs logiques et opérateurs de comparaisons

Pour rappel, il n'est pas demandé de gérer les parenthèses, l'opérateur unaire `-`, l'opérateur de concaténation `&`, etc. mais vous pouvez les implémenter comme bonus.

Livrables et échéances

Pour le **prochain cours (semaine du 03/02)**, vous établirez une **première version du diagramme de classes du domaine** pour les fonctionnalités de cette itération et vous le pousserez dans le dépôt de votre projet. Lors de ce cours, le chargé de cours commentera de ce modèle avec vous.

Pour le **dimanche 16/03** à 23h59, **remise du code de l'itération 1** dans votre dépôt, ainsi qu'un **diagramme de classes logiciel** reprenant les éléments importants de votre conception, sans détails inutiles, expliquant les liens entre les classes du modèle, les *view-models* et les vues.